

Deep RL in Action — Learning Journal

A chapter-by-chapter conclusion for the "Deep Reinforcement Learning in Action" book, based on the notebooks under `pytorch_cspo/`.

Overview

The repo covers Chapters 2 through 10 of the book (skipping Chapter 6). Each chapter has one or more notebooks implementing the algorithm from scratch, often extended beyond the book — for example the multi-agent notebook goes past the book to deploy the algorithm in the `magent2 battle_v4` environment. This note summarizes what each notebook actually implements, notes worth keeping, and — for Chapter 10 — the debugging story that took a fair amount of investigation to resolve.

Chapter 2 — Multi-Armed Bandits

Notebooks: `bandits_arm.ipynb`, `bandits_arm_env.ipynb`.

The first bandit notebook is pure NumPy: 10 arms with Bernoulli success probabilities, softmax action selection (temperature 1.12), sample-average value updates. No neural net. The second notebook builds a **contextual bandit** — 10 states × 10 arms, uniform state transitions — and uses a 10→100→10 MLP to map one-hot state to per-arm Q-values, with softmax exploration and MSE regression toward `target = current_logits[except arm] , target[arm] = reward`.

Key idea to remember: the "target = logits with only one value replaced" trick keeps unselected actions from being pushed anywhere. That same pattern reappears in every DQN-style method downstream.

Chapter 3 — DQN on Gridworld

Notebooks: `gridworld.ipynb`, `practice.ipynb`.

`gridworld.ipynb` is the primary implementation: a custom 4×4 grid environment with player, goal, pit, and wall pieces (`mode='static'`), a 64→150→100→4 MLP, replay buffer (size 5000), target network synced every 100 gradient steps, ϵ -greedy annealed 1.0→0.1. Notable trick: `to_state` adds 0.1 uniform noise to the one-hot board representation to break symmetry between visually similar states. `practice.ipynb` is a second-pass re-implementation on the harder `mode='random'` and `mode='player'` variants, running DQN both with and without a target network for direct comparison.

Key idea to remember: the target network is what stabilizes Q-learning. Without it, you're chasing a moving target; with it, the regression targets change slowly enough that gradient descent can catch up. The demo of "same DQN, with vs without target network" in `practice.ipynb` makes this concrete.

The trailing `test_model()` in `gridworld.ipynb` has a small bug (`render_np.reshape` without parens) — worth fixing if returning to this notebook.

Chapter 4 — Policy Gradients

Notebook: `policy_gradient_descent.ipynb`.

REINFORCE on CartPole-v1 with a $4 \rightarrow 100 \rightarrow 2$ softmax policy net and full discounted normalized returns. Loss is the classic $-\sum \log \pi(a|s) \cdot G_t$. 500 episodes, `MAX_DURATION=200`, Adam $lr=9e-3$. A final human-render loop runs 5 greedy episodes.

Key idea to remember: unlike Q-learning, policy gradients directly optimize the policy — no target network, no bootstrap. The reward-baseline (normalized returns) is what keeps variance manageable. This is also the first place `torch.distributions.Categorical` shows up.

Chapter 5 — Actor-Critic / A2C

Notebooks: `DA2C.ipynb`, `n-step_DA2C.ipynb`, `bootstrapping.ipynb`.

`DA2C.ipynb` implements distributed A2C with 15 worker processes sharing model parameters via `mp.share_memory()`. The `ActorCritic` net splits into an actor head (log-softmax over 2 actions) and a critic head (scalar value, tanh). Workers run full-episode Monte Carlo rollouts, no bootstrap, with critic coefficient `clc=0.1` and terminal return -10. There's a note on Python 3.14's default forkserver being overridden to `fork` for CUDA compatibility. `n-step_DA2C.ipynb` swaps in truncated rollouts of length `N_steps=10` with bootstrap `G = value.detach()` — this is the same n-step return idea that ends up being the key fix in Chapter 10. `bootstrapping.ipynb` is a tiny didactic snippet comparing MC and bootstrapped returns on toy reward streams `[1, 1, -1]` vs `[1, 1, 1]`.

Key idea to remember: the actor-critic split lets you use TD(0) or n-step bootstrapping to reduce variance (relative to REINFORCE), while keeping the policy-gradient-based update. And parallelizing rollouts across workers gives you IID-ish samples without the replay buffer cost.

Chapter 6 — Evolutionary Algorithms

Not implemented in this repo. No notebook in `pytorch_cspo/` corresponds to Chapter 6. If revisiting: the book implements CEM-style evolution on top of the same primitives already coded in Ch4/Ch5. `torch.randn(size) / 10.0` for parameter init (from `old_but_more_detailed/Ch6_book_dev.ipynb`) is the pattern the book uses to manage parameters as a flat tensor rather than an `nn.Module`.

Chapter 7 — Distributional DQN

Notebooks: `Dist_DQN.ipynb`, `Dist_DQN2.ipynb`.

C51-style distributional DQN on `ALE/Freeway-v5` with 128-dim RAM observations and 3 actions (NOOP/UP/DOWN). Uses 51 atoms on support `[-10, 10]`. Interesting detail: the network is built with manual weight matrices (`theta`) rather than an `nn.Module` — 128→100→25 backbone then per-action 25→51 head. `get_target_dist` handles the categorical-projection step: on terminal reward it collapses to a one-hot at the observed return, otherwise it shifts the distribution using $r + \gamma \cdot Z$. Priority replay is implemented by duplicating positive-reward transitions ×4 into the deque. The final cell animates frames alongside the per-action $Z(s,a)$ distributions — very readable diagnostic. `Dist_DQN2.ipynb` is nearly identical except it bootstraps from `state_batch` instead of `state2_batch` — likely an experiment or a bug worth flagging.

Key idea to remember: predicting a return *distribution* instead of an expected value gives the network more shape to fit — the training signal is richer, and visualizing $Z(s,a)$ makes agent uncertainty inspectable. The categorical projection (`get_target_dist`) is the trickiest piece and worth re-deriving on paper if returning to this.

Chapter 8 — Curiosity-Driven Exploration / Super Mario

Notebooks: `curiocity.ipynb`, `curiocity_gpu.ipynb`.

Full ICM (Intrinsic Curiosity Module) on `SuperMarioBros-v0` via `nes-py` + `gym_super_mario_bros`, using `JoypadSpace(COMPLEX_MOVEMENT)` for 12 discrete actions. Frames are downsampled to 42×42 grayscale and stacked (3 frames). Three networks: encoder Phi (4 stride-2 convs → 288-d features), inverse model Gnet (576→256→12 logits) that predicts action from $(\phi(s), \phi(s'))$, and forward model Fnet (300→256→288) that predicts $\phi(s')$ from $(\phi(s), a)$. Intrinsic reward is the forward-model prediction error; the inverse-model loss constrains Phi to encode only what actions can influence. Separate DQN Q-network on top. `curiocity_gpu.ipynb` is the same code with a `device = torch.device(...)` block added but — as commented — `.to(device)` was never actually applied to the models/tensors, so it still runs on CPU. Both notebooks have known bugs annotated in the source (e.g. `action_.to(state.device)` missing).

Key idea to remember: intrinsic reward from forward-model surprise gives dense supervision in sparse-reward tasks. But surprise about "things you can't control" (background noise) is a distractor — the inverse-model auxiliary loss is what removes uncontrollable dimensions from the feature space.

Chapter 9 — Multi-Agent RL / Ising Model

Notebook: `MARL_Isingmodel.ipynb`.

The most ambitious notebook in the repo. Three progressive environments: (1) a custom 1D Ising grid of size 20, (2) a 2D Ising grid 10×10 with mean-field neighborhood aggregation, and (3) the `magent2_battle_v4` environment with two 20×20 teams (red vs blue). The algorithm is **mean-field Q-learning**: each agent has its own parameter vector `theta` (or a shared one for the 2D case), Q-function takes `(joint state, mean neighbor action)` as input, softmax exploration. For the harder 2D and battle environments the code adds a replay buffer, `gamma=0.9`, TD target `r +  $\gamma \cdot \max Q(next)$` . The battle deployment extends beyond the book: alive-agent id tracking after `raw.clear_dead()`, per-team parameter sets, minimap-to-RGB frame recording, and an animated rollout. Explicit before/after `sum(theta)` diagnostics verify that in-place parameter updates are actually happening.

Key idea to remember: mean-field approximation reduces "N agents each seeing N-1 others" to "each agent seeing the average of its neighbors" — a huge simplification that keeps the problem tractable. The book stops at 2D Ising; the notebook shows the same approximation carrying through to a real MARL environment.

Chapter 10 — Attention / Relational RL / MiniGrid

Notebooks: `attention.ipynb`, `Multihead.ipynb`, `Multihead_FullyObs.ipynb`, `Multihead_DotProduct.ipynb`, plus follow-up experiment scripts under `pytorch_cspo/*.py`.

`attention.ipynb` builds a single-head relational module on augmented MNIST — random affine, rotation, salt noise — as a pedagogical warm-up. 4 stride-1 convs produce 16×16=256 "nodes"; 2D spatial coordinates are concatenated; standard scaled dot-product attention ($Q \cdot K^T / \sqrt{d}$, softmax); Linear → max-pool → classification head, NLL loss. `self.att_map` is saved for inspection.

`Multihead.ipynb` applies the same idea to `MiniGrid-DoorKey-5x5-v0`. Two 1×1 convs plus spatial coordinates, then 3 attention heads of 64-d Q/K/V. This is where the interesting story starts. The book uses **additive attention** — `A = softmax(a_lin(elu(k_lin(K) + q_lin(Q))))` — rather than the standard dot-product form. `Multihead_DotProduct.ipynb` is an ablation that swaps in dot-product attention. `Multihead_FullyObs.ipynb` swaps `ImgObsWrapper` for

`FullyObsWrapper` so the network sees the entire 5×5 map instead of the 7×7 partial view.

The debugging story (Chapter 10, from the notebooks)

Initial symptom: `Multihead.ipynb` trained cleanly (loss dropped from ~0.05 to ~0.001) but greedy evaluation was consistently 0% success. All Q-values collapsed to nearly the same value across all states — a policy-invariant, essentially-constant Q function.

What we tried and mostly ruled out (44 architectural trials across two overnight sweeps under `sweep_results/` and `sweep_results_v2/`):

- Additive → scaled dot-product attention
- Removing the `elu` on the Q output
- Max pool → mean pool
- LayerNorm `elementwise_affine=True`
- ϵ annealing 1.0 → 0.05 vs fixed 0.5
- Reward duplication (N=50, N=100, N=1)
- Learning rate 5e-4 vs 1e-4
- Target update frequency 100 vs 500
- Observation normalization by max value or by 10
- Fully-observable vs partial view
- Longer training (30k → 60k epochs)
- Weight regularization on unselected Q-values
- Shrinking initial weights ×0.1

None of these reliably fixed the collapse. Some helped on lucky seeds (26.7%, 40%, 100% — coin flips), but seeded runs weren't reproducible across launches because CUDA non-determinism dominated.

Two things did matter:

- 1 `_shrink_init(scale=0.1)` — shrinking initial weights ten-fold reliably killed training. Weight initialization at that scale drives conv activations near zero, which combined with LayerNorm's normalization prevents the network from ever escaping the "predict the mean" attractor. Removing this alone (or using default init) is necessary but not sufficient.

- 1 **The MSE loss function had a broadcasting bug.** The original:

```
python def lossfn(pred, targets, actions): return torch.mean(torch.pow(
targets.detach() - pred.gather(dim=1, index=actions.unsqueeze(dim=1)), 2
))` Here targets has shape (batch,) but pred.gather(...) has shape (batch, 1).
Subtracting them produces the outer difference of shape (batch, batch) —
every target minus every prediction. The optimal fit under this loss is
predicting the batch mean of targets for every state, which is exactly
```

```
the constant-Q collapse we were seeing. The MSE loss made the collapse mathematically optimal. The fix is one .squeeze(1): `python q_sa = pred.gather(dim=1, index=actions.unsqueeze(dim=1)).squeeze(1) loss = torch.mean(torch.pow(targets.detach() - q_sa, 2)) ``
```

With the broadcasting bug fixed and `_shrink_init` removed, the book's original architecture (additive attention, `elu` on Q, non-affine LayerNorms) trains successfully. No architectural changes are actually required — the collapse story from the sweep was tracking symptoms of the two setup bugs above.

The separate n-step finding

For a *cleanly-implemented* DDQN (no lossfn bug, no shrink), 1-step Bellman targets on DoorKey are a coin flip — the value front doesn't propagate back through the ~15-step chain (navigate → pickup key → navigate → toggle door → reach goal) fast enough. Switching to n-step returns (n=3 is the sweet spot) turns eval success into a near-deterministic 100%. The reason: with n=3 targets, five Bellman updates worth of directly-observed reward are baked into each training target, so value estimates for early states in the sequence are grounded in observation rather than in noisy chained Q estimates.

Key ideas to remember from Ch10:

- 1×1 convs are per-cell linear projections; the *attention* module is what mixes information across spatial positions. This is a fundamentally different inductive bias than standard CNNs and worth internalizing.
- Adding normalized spatial coordinates as extra channels is the trick that lets attention learn spatial relationships even though its projections are position-agnostic.
- The book's additive attention formulation is unusual; scaled dot-product attention (as in standard Transformers) is essentially interchangeable here.
- **Always print tensor shapes before subtracting in a loss** — the broadcasting bug hid in plain sight for 40+ trials of wrong hypotheses. `assert q_sa.shape == targets.shape` or `F.mse_loss` (which errors on mismatch) would have caught it immediately.
- For sparse-reward long-horizon tasks, n-step returns are worth the small amount of trajectory-buffering bookkeeping.

Related scripts under `pytorch_cspo/`:

- `test_nstep.py` — clean n-step DDQN reference (~93-100% success reliably)
- `test_book_vs_corrected.py` — architectural ablation, book vs "corrected" model at n=2 and n=3
- `test_notebook_repro.py` — reproduces the collapse via the two bugs above (shrink init + N=100)
- `overnight_sweep.py`, `overnight_sweep_v2.py` — the 32-trial architectural search (mostly negative)

- `diagnose.py` — the 4-test diagnostic ladder (Q-spread, gradient flow, tiny-buffer overfit, CNN baseline)
 - `MORNING_REPORT.md`, `FINAL_REPORT.md` — running write-ups of the debugging session
-

Miscellaneous

Pre-book tutorials: `torch_rl.ipynb` sketches PyTorch RL primitives (`nn.Sequential`, `torch.distributions.Categorical`, `.log_prob`, `.entropy`, `.backward()`) — good as a prelude to Ch4. `module_tutorial.ipynb` explores the `gym_super_mario_bros` and `gymnasium` module APIs — prep work for Ch8.

Duplicates and variants:

- `gridworld.ipynb` and `practice.ipynb` both implement Ch3 DQN (the second is a re-do on harder settings)
 - `curiosity.ipynb` and `curiosity_gpu.ipynb` are near-duplicates; the "gpu" version is aspirational — the `.to(device)` calls were never actually applied
 - `Dist_DON.ipynb` and `Dist_DON2.ipynb` differ in one line (bootstrap from `state_batch` vs `state2_batch`) — the second is either an ablation or a bug
-

Suggested next steps

- **Chapter 6** (Evolutionary) is the missing chapter; implementing it against a small CartPole target would round out the book coverage
- **Fix known bugs:** `test_model()` in `gridworld.ipynb`, GPU migration in `curiosity_gpu.ipynb`, revisit whether `Dist_DON2.ipynb`'s single-line diff was intentional
- **Consolidate the Ch10 findings** into a cleaner reference notebook that combines: original book architecture + fixed `lossfn` + n-step targets + a proper eval protocol
- **Replay these algorithms against a common environment set** (CartPole, MiniGrid, Freeway, Mario) side-by-side to build intuition for when each family works best — the current organization is per-chapter, not per-environment, which makes cross-comparison harder